

CodeGen RAG Capstone

Engineering, Synchronization & Delivery Report

A complete record of the engineering work performed on the project — repository synchronization, bug fixes, testing, publishing, and final deliverable preparation

AIML PGCP Capstone

Group Number: 39 · Batch: 26

TalentSprint / Accenture

Team Members:

Mahin Nandipa · Abhinaya Thavishi · Ashu Bagul

Mentors: Pawan Baswani, Nayan Anand

Supervisor: Prof. Anil Neelkanti

July 2026

Executive Summary

This report documents the full body of engineering work carried out on the CodeGen RAG Capstone project beyond the original four-checkpoint research notebooks: verifying and repairing synchronization between exploratory sandbox work and the team's live Google Drive repository, diagnosing and fixing three infrastructure-level bugs unique to running Colab against a Drive-mounted repo, closing two functional gaps (checkpoint-aware model routing and genuine AST-aware CodeBLEU scoring), growing the automated test suite from 108 to 147 passing tests, publishing the repository to GitHub twice (a personal repository and a shared mentor collaboration repository), consolidating project documentation into a comprehensive README, and restructuring the three final academic deliverables (Technical Report, Presentation Slides, Research Paper Summaries) to match the official capstone rubric.

Every fix described here was verified before being reported as complete: file writes were confirmed via MD5 hash read-back, the test suite was re-run and its pass count reported by the user at each stage, and GitHub pushes were confirmed against the live repository page. Where something remains incomplete or unverified, this report says so explicitly rather than implying otherwise — see Section 8, Outstanding Items.

1. Project Background — What Already Existed

Before this engagement's engineering work began, the CodeGen RAG Capstone already existed as a complete, four-checkpoint research project: a system for evaluating and enhancing Salesforce/codegen-350M-multi, a small open-source code language model, across five software-engineering tasks (program synthesis, documentation generation, commit message generation, code translation, and natural-language-to-SQL generation), compared against a Rust-fine-tuned variant, an upper-bound LLM (Claude Sonnet 4 with GPT-5 fallback), and a Retrieval-Augmented Generation pipeline layered on top of that upper-bound LLM.

1.1 Existing Repository Structure

The project lived in a Google Drive-mounted Colab repository at codegen-rag-capstone, organized as a typed-configuration Python package (`src/codegen_rag/`) with submodules for data downloading and preprocessing, model wrapping and a model registry, five task modules built on a common `BaseTask`, an evaluation harness, a hybrid FAISS/AST retrieval pipeline, and a FastAPI + Streamlit deployment layer, alongside four checkpoint notebooks (one per project phase) and a pytest test suite.

1.2 Existing Functionality Prior to This Engagement

- Checkpoint 1: environment bootstrap, CoDocBench + Rust corpus download, baseline evaluation of the untouched base model on four tasks, and a first Rust LoRA adapter trained on a 500-sample subset.
- Checkpoint 2: Spider + BIRD download with SQLite database archives, schema-aware SQL generation, execution-accuracy scoring, and a full-corpus Rust fine-tune.
- Checkpoint 3: a 2,000+ sample retrieval corpus, a FAISS dense index and a separate AST structural index, a Top-K/strategy sweep, and the project's first four-tier model comparison.
- Checkpoint 4: a FastAPI backend exposing every task and the RAG pipeline, a four-tab Streamlit demo UI, and Docker/docker-compose containerization.
- An existing pytest suite (108 tests at the start of this engagement) covering the above with fakes standing in for model, network, and filesystem calls.

This existing work was the starting point. Everything described in Sections 2 through 6 below is engineering performed during this engagement on top of that foundation — either fixing something that was broken, adding something that was missing, or preparing the project for external delivery.

2. Repository Synchronization — Sandbox vs. Live Drive Repo

This engagement's engineering phase began when the team asked whether changes made in an exploratory sandbox environment had actually reached their live Google Drive repository — the answer, on investigation, was mostly no, and two further problems were uncovered in the process.

2.1 The Repo-Path Mismatch

`settings.root_dir`, resolved by the project's `bootstrap_environment()` function from `configs/config.yaml`'s `drive_subdir` setting, pointed to `/content/drive/MyDrive/CodeGen_Capstone` — a data root used for checkpoints, logs, and results. The actual git-tracked source-code repository, however, lived in a different Drive folder entirely: `/content/drive/MyDrive/codegen-rag-capstone`, confirmed by reading the `rootdir` path `pytest` itself printed at the top of its output. Every subsequent patch cell in this engagement targets the second, confirmed-correct path directly, with a `pyproject.toml` existence check as a guard rail, rather than trusting `settings.root_dir`.

2.2 Test Count Investigation (108 → 125 → 147)

An initial sync pass brought three files (LICENSE and two test files) up to date, taking the suite from 108 to 125 passing tests. Comparing the sandbox's `pytest --collect-only -q` output line-by-line against the team's own pasted verbose test run identified exactly six files with twenty-two still-missing tests. The team chose a full synchronization pass, delivered as a single combined Colab cell (`patch_full_sync.py`) carrying four source files and six test files, taking the suite to 147 passing tests — its current state as of this report.

2.3 Files Synchronized in the Full-Sync Pass

- `src/codegen_rag/models/codegen_wrapper.py`
- `src/codegen_rag/models/model_registry.py`
- `src/codegen_rag/evaluation/evaluator.py`
- `src/codegen_rag/data/preprocessors.py`
- `tests/test_api.py`, `tests/test_api_client.py`, `tests/test_evaluator_sql.py`, `tests/test_preprocessors.py`, `tests/test_codegen_wrapper.py`, `tests/test_model_registry.py`

Every file in every patch cell throughout this engagement was written using the same pattern: base64-encode the verified sandbox content, write it to the target path, wait, read it back, and compare its MD5 hash against the expected value — retrying up to five times before failing loudly — because of the Drive write-sync race described in Section 4.1.

3. Functional Engineering Fixes

Two functional gaps were identified and closed during deployment hardening, beyond the original four-checkpoint research scope.

3.1 Checkpoint-Aware Model Routing

Every deployment endpoint had been silently serving the untouched base model regardless of whether a trained Rust checkpoint existed on disk — the routing logic that was supposed to prefer a fine-tuned checkpoint was never actually finding one. The root cause was a checkpoint-directory nesting bug (detailed in Section 4.3): the lookup checked one directory level too shallow. This was corrected with a resolution layer — `_best_checkpoint_dir()`, `_resolve_run_checkpoint()`, and `_is_valid_checkpoint()` — that locates the best available checkpoint per language, preferring the run's own recorded lowest-loss checkpoint from `training_state.json` and falling back to the highest-numbered saved checkpoint. The fix is covered by eight dedicated unit tests and was confirmed live in the Streamlit demo, which now reports "rust: fine-tuned checkpoint active (checkpoint-002200)" rather than silently using the base model.

3.2 Genuine AST/Dataflow-Aware CodeBLEU

CodeBLEU is designed to combine weighted n-gram overlap with AST structural match and dataflow match. Without a Python grammar available to the codebleu package, its AST and dataflow components were silently falling back to a plain sacrebleu text-similarity approximation — no error was raised, so the scores looked plausible while measuring the wrong thing. Adding tree-sitter-python==0.21.0, a version explicitly verified compatible with the project's pinned tree-sitter==0.22.3 / codebleu==0.7.0 combination (newer tree-sitter-python releases raise a TypeError due to an ABI-breaking change in the Language() binding introduced in 0.23.0), restored genuine structural and dataflow scoring.

3.3 LICENSE and .gitignore

An MIT LICENSE file (crediting Group 39, Batch 26 collectively) was added to the repository, having been absent. Separately, and more significantly, a .gitignore file that had been assumed present and "verified adequate" in an earlier sandbox check turned out to be completely absent from the actual Drive repository — this was the direct cause of the __pycache__ contamination described in Section 4.2 and Section 5. A 39-line .gitignore covering data/checkpoint/model exclusions, secrets, Python build artifacts, notebook checkpoints, and OS files was written and committed.

4. Infrastructure Issues Diagnosed and Resolved

Running and patching this project live against a Google-Drive-mounted Colab repository surfaced three issues that were not code bugs in the ordinary sense — each looked, at first, like a regression in the application code, and each required root-causing before the real, infrastructure-level explanation became clear.

4.1 Google Drive FUSE Write-Sync Race

A file write to a Drive-mounted path can succeed and self-verify — an immediate read-back inside the same Colab session matches the intended content — without having actually flushed to persistent Drive storage. A subsequent Runtime restart can then silently revert the file to an older cached version, with no error or warning at any point. This was root-caused through repeated byte-count and MD5 re-verification of a single patched file across multiple restarts. The operational fix, applied to every patch cell in this engagement from that point forward, is a five-attempt write-then-verify loop: write the file, wait 1.5 seconds, read it back, compare its MD5 hash to the expected value, and retry if it doesn't match, raising an error only after five failed attempts.

4.2 Stale Compiled Bytecode on a Drive-Mounted Package

Python decides whether to reuse a cached .pyc bytecode file by checking its source .py file's reported modification time. Google Drive's FUSE mount can report modification times unreliably, causing Python to keep executing an old compiled version of a module even after the corresponding source file on disk was directly confirmed correct — this surfaced as an ImportError for a function whose source text was demonstrably present in the file being imported. The fix was to recursively delete every __pycache__ directory under the package before the next Runtime restart, forcing a clean recompilation from the current source.

4.3 Checkpoint Directory Nesting Bug

The checkpoint-routing feature (Section 3.1) initially failed because its lookup checked for checkpoint marker files one directory level too shallow — the project's CheckpointManager.save() nests every save under a checkpoint-NNNNNN subdirectory rather than writing directly into the named run directory. This was only discovered by checking the actual on-disk layout of the team's already-trained Rust checkpoints in response to the team's question of whether earlier checkpoints needed to be retrained; they did not — the checkpoints already existed, and the lookup logic simply was not finding them. Fixed with an explicit checkpoint-resolution helper and eight dedicated unit tests.

5. Testing

The automated test suite grew from 108 passing tests at the start of this engagement to 147 passing tests by its end, entirely through the synchronization and bug-fix work described in Sections 2 and 3 — no test was weakened or skipped to reach this number. Every test runs without a GPU, live network access, or downloaded model weights, using fakes at every heavy call site (FakeCodeGenModel / FakeAPIModel for generation and embedding calls, FastAPI dependency_overrides for endpoint tests, and on-disk tmp_path fixtures for the checkpoint-resolution tests). The __pycache__ contamination discovered in Section 4.2 and the missing .gitignore discovered in Section 3.3 were caught specifically because running this expanded suite against the live Drive repo revealed a large, unexpected set of untracked and then wrongly-tracked bytecode files.

6. Version Control and Publishing

6.1 Git Identity and First Commit

Git identity (user.email and user.name) had to be configured per-session, since Colab runtimes are ephemeral and do not persist global git config across restarts even though the repository itself lives on Drive. The first commit attempt failed with "Author identity unknown" until this was set. The first successful commit swept up 166 files, including — unintentionally, per Section 4.2 and 4.3 — a large number of __pycache__ bytecode files, which were subsequently removed once the missing .gitignore was identified and added, via git rm -r --cached --ignore-unmatch and a follow-up cleanup commit (84 files changed).

6.2 Personal GitHub Repository

The repository was pushed to a new personal GitHub repository (mahin-aeroai/CodeGen-). The first push attempt failed with a 403 "Permission... denied" error; this was diagnosed as insufficient scope on the GitHub Personal Access Token rather than a missing or nonexistent repository (confirmed by inspecting the empty-but-existing repository page directly). Regenerating a classic PAT with the top-level repo scope checkbox explicitly checked resolved the push. The token was scrubbed from git remote -v immediately after each successful push via git remote set-url origin <clean-url>, so it is never left sitting in .git/config.

6.3 Mentor Shared Collaboration Repository

The team was also given a shared, multi-group mentor repository (nayanjha16/CodeGen-Implementations-May_26) with an existing Group-39 branch already containing a teammate's prior commits. Rather than merging two independent git histories directly (which would have triggered an "unrelated histories" rejection on push), the Group-39 branch was cloned separately and the project's files were copied into it with shutil.copytree(..., dirs_exist_ok=True), preserving the existing branch history and non-colliding files. Per the team's explicit instruction that this is a group effort, the push used generic, name-free git identity (user.name "Group 39") and commit messages, and generically named files (e.g. README_implementation.md rather than a personally-named file) — while being transparent that GitHub's authenticated push account remains visible in commit metadata regardless of the display name used, since PATs are account-scoped rather than repo-scoped and the same token that worked for the personal repository also had push access here.

6.4 README Consolidation

The mentor repository's existing team README.md (a short table of team members) was kept intact rather than replaced, per the team's instruction. The full, comprehensive project documentation was merged directly into that same default-displayed README.md — rather than left as a separate linked file — so that browsing the Group-39 branch shows complete project documentation by default, matching the depth of a peer team's (Group-19) README shown as a reference example. The merge cell used is idempotent: it strips any earlier-added link-marker section before appending the full content, so it can be safely re-run without duplicating content, and it removes the now-redundant standalone implementation file once merged.

7. Final Capstone Deliverables

Three final deliverables were produced and, on the team's request, restructured to explicitly match the official capstone submission rubric (title page with title, group number, and team names; problem statement; literature review; methodology; outcomes in stages; final outcomes; challenges; applicability in the real world).

7.1 Technical_Report.docx

An eight-page Word document ordered exactly to the rubric: Title Page, Abstract, Problem Statement, Literature Review, Methodology, Outcomes in Stages (new — a per-checkpoint-stage breakdown of concrete outputs, plus a dedicated subsection for the post-deployment engineering outcomes in Sections 3–4 of this report), Final Outcomes (the baseline comparison table and discussion), Challenges (the three infrastructure issues from Section 4 of this report, reframed for an academic audience, plus scope/resource constraints), Applicability in the Real World (new — six subsections on practical use cases), Limitations and Honest Scope Note, Conclusion and Future Work, and References.

7.2 Presentation_Slides.pptx

A twenty-slide deck (grown from an earlier eighteen-slide version) in the project's existing "Midnight Executive" visual style, with two new slides added — Literature Review and Applicability in the Real World — and the four checkpoint slides and results slides relabeled to the rubric's exact terminology ("Outcomes in Stages · Stage 1–4" and "Final Outcomes"). Full speaker notes were added to every slide, since the team asked for a "full explanation presentation." The deck passed automated structural validation and was rendered to images for visual quality-assurance review before delivery.

7.3 Research_Paper_Summaries.docx

Detailed summaries of all twelve papers cited in the project README, with two citation corrections identified during preparation (CoDocBench's author is Pai, not "Pal" as the README states; BIRD was published in 2023, not "2024" as the README states) and one citation — "AST retrieval (Jiang et al., 2023)" — flagged as unverifiable against a specific matching publication rather than silently left uncorrected or fabricated a source for.

8. Current State Summary

The table below is a single source of truth for what has been completed, verified, and delivered versus what remains outstanding as of this report.

Item	Status	Notes
Drive repo synced to sandbox-verified state	Done — 147/147	Confirmed by team's own pytest run output.
Checkpoint-aware model routing	Done	Verified live in Streamlit demo + 8 unit tests.
Genuine tree-sitter CodeBLEU	Done	Verified via pinned-version compatibility check.
LICENSE	Done	MIT, group-credited, MD5-verified on Drive.
.gitignore	Done	Was entirely missing; now 39 lines, committed.
Push to personal GitHub repo	Done	mahin-aeroai/CodeGen-, main branch.
Push to mentor shared repo	Done	nayanjha16/.../Group-39 branch, group-attributed.
README consolidation on Group-39 branch	Handed off	Merge cell provided; not yet confirmed run by team.

Item	Status	Notes
Technical_Report.docx (rubric-aligned)	Done	8 pages, all rubric sections present.
Presentation_Slides.pptx (rubric-aligned)	Done	20 slides + full speaker notes, validated.
Research_Paper_Summaries.docx	Done	12 papers, 2 citation corrections, 1 flagged.
Full end-to-end GPU run (fine-tuned/LLM/RAG tiers)	Pending	Only small-LM baseline tier has real numbers so far.

9. Outstanding Items and Next Steps

- Confirm the README consolidation cell was actually run on the mentor repo's Group-39 branch — this was handed off but not yet confirmed executed.
- Run the full four-notebook sequence end to end in Colab with a GPU and live API keys to populate the fine-tuned Rust, upper-bound-LLM, and RAG-augmented tiers of the comparison table — only the small-LM baseline tier has real, non-placeholder numbers as of this report.
- Reproduce and resolve the PEFT "missing adapter keys" warning observed when loading the Rust LoRA checkpoint.
- Resolve the unverified "AST retrieval (Jiang et al., 2023)" citation against the original project proposal or literature search before external submission.